

Installer des logiciels :

1. VS Code

- Aller sur Firefox
- Télécharger le fichier pour Linux Ubuntu (code_blabla.deb) depuis le site Vs Code
- mettre `sudo apt install ./<file>.deb` et <file> c'est le nom du fichier (tips : tab+fleche pr trv le fichier)
- Aller sur le site <https://code.visualstudio.com/docs/setup/linux>
- mettre `echo "code code/add-microsoft-repo boolean true" | sudo debconf-set-selections`
- mettre `sudo apt install apt-transport-https`
- mettre `sudo apt update`
- mettre `sudo apt install code`
- mettre `code .` pour lancer VS Code !!

2. Unreal Engine 5

- on a fait la commande `df -h` et on s'est rendu compte qu'on aurait pas assez de place pour ce logiciel

3. Blender

- mettre `sudo apt install blender`
- mettre `blender` pour lancer blender
- le logiciel crash et met des messages d'erreur dans le terminal car la machine n'était pas assez puissante (pas assez de RAM)

4. GIT

- mettre `sudo apt install git`
- mettre n'importe quelle commande git et ça marche (exemple `git -v` pour voir la version)
- (voir fiche sur git pour d'autres commandes)

Pourquoi on a choisi ces logiciels ?

vscode: léger et on peut mettre des extensions pour dev en c

blender: incontournable de la création d'assets 3D

git: pour le versionning et le développement en équipe

5. GCC Compilateur

- mettre `sudo apt install gcc`
- mettre `gcc -v` pour voir la version et si tout est ok

Le programme C :

-mkdir prog1 et cd prog1

-nano code1.c

```
#include <stdio.h>

int main() {

float numerateur, denominateur, resultat;

printf("Entrez le numérateur : ");

scanf("%f", &numerateur);
printf("Entrez le dénominateur : ");

scanf("%f", &denominateur);

if (numerateur < 0 || denominateur < 0)

    {

        printf("Erreur : Les nombres négatifs ne sont pas autorisés.\n");

        return 2;

    }

if (denominateur == 0)

    {

        printf("Erreur : Division par zéro impossible.\n");

        return 1;

    }

resultat = numerateur / denominateur;

printf("Le résultat de %.2f divisé par %.2f est : %.2f\n", numerateur, denominateur, resultat);

return 0;

}
```

-on va compiler le fichier c (pr le mettre en binaire et que ce soit exécutable) avec la commande `gcc -o sortie1.elf code1.c`

elf = convention des fichiers exécutables sur Linux

-on doit mettre les perms d'exécutions sur le fichier `chmod 777 sortie1.elf`

-on exécute le programme avec `./`

`./sortie1.elf`

-le programme il marche correctement !

Script Bash pour tester le programme en c :

-le premier programme n'était pas compatible à cause des entrées clavier (scanf), faut que les nombres à entrer soit en argument, donc on va modifier le code en utilisant des fonctions qu'on a prit sur la bibliothèque <stdlib.h>
du coup on modifie le code pour que le numérateur et dénominateur soit des arguments (à récupérer dans argc ou argv)

-cd ../ puis mkdir prog2 puis cd prog2 puis nano code2.c

```
#include <stdio.h>
#include <stdlib.h> //atof // argc // argv
int main(int argc, char *argv[]) {
float numérateur, dénominateur, resultat;
if (argc != 3)
{
printf("Usage : %s <numérateur> <dénominateur>\n", argv[0]);
return 1;
}
numérateur = atof(argv[1]);
dénominateur = atof(argv[2]);
if (numérateur < 0 || dénominateur < 0)
{
printf("Erreur : Les nombres négatifs ne sont pas autorisés.\n");
return 2;
}

if (dénominateur == 0)
{
printf("Erreur : Division par zéro impossible.\n");
return 1;
}
resultat = numérateur / dénominateur;
printf("Le résultat de %.2f divisé par %.2f est : %.2f\n", numérateur, dénominateur, resultat);
return 0;
}
```

-gcc -o sortie2.elf code2.c

-chmod 777 sortie2.elf

-pour exécuter le nv code : ./sortie2.elf <numérateur> <dénominateur> !!!

atof ca convertit en float les arg exemple : ./sortie 12 6
*argc contient 3 car il y a 3 argument (séparation par l'espace) *
argv va donc contenir: ['./sortie', '12', '6']

Maintenant on va faire le script bash qui va tester le programme c :

-dans prog2, faire nano test_script.sh

```
#!/bin/bash
echo "Début des tests du programme de division"
echo "-----"
```

```
echo -e "\nTest 1: Division normale"
./sortie2.elf 10 2
if [ $? -eq 0 ]; then
    echo "Test 1 réussi"
else
    echo "Test 1 échoué"
fi
```

```
echo -e "\nTest 2: Division par zéro"
./sortie2.elf 10 0
if [ $? -eq 1 ]; then
    echo "Test 2 réussi"
else
    echo "Test 2 échoué"
fi
```

```
echo -e "\nTest 3: Numérateur négatif"
./sortie2.elf -10 2
if [ $? -eq 2 ]; then
    echo "Test 3 réussi"
else
    echo "Test 3 échoué"
fi
```

```
echo -e "\nTest 4: Dénominateur négatif"
./sortie2.elf 10 -2
if [ $? -eq 2 ]; then
    echo "Test 4 réussi"
else
    echo "Test 4 échoué"
fi
```

```
echo -e "\nTest 5: Absence d'arguments"
./sortie2.elf
if [ $? -eq 1 ]; then
    echo "Test 5 réussi"
else
    echo "Test 5 échoué"
fi echo -e "\nFin des tests"
```

-e permet d'utiliser \n

\$? c'est ce qui est retourné par ./sortie2.elf bla bla

-eq c'est comme un = sauf que sous linux, le vrai = c'est pour les chaînes de caractères

-pour le rendre exécutable `chmod +x test_script.sh`

-pour l'exécuter faire `./test_script.sh` et **gg le test marche youhou c'est la fête**